ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем
МДК.02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №17»

Выполнил:	Напалков Глеб Васильевич
Студент:	54-15 группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководитель:	Борисов Егор Николаевич
Оценка:	_____

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА

КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ
Заведующий отделением
информационных технологий
А.Ю. Бычков
_____
«____» _____ 2026 г.

ЗАДАНИЕ

для выполнения курсового проекта

Специальность: 09.02.08 «Интеллектуальные интегрированные системы»

Группа: 54-15

Ф.И.О. студента: Напалков Глеб Васильевич

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №17»

Разработка принципиальной схемы в EasyEDA, модели Arduino Uno в Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED; ведение проекта в Git.

Входные данные варианта	**Значение**
Сценарий	Кухня ресторана
Датчик 1 (аналоговый)	MQ2 — газ, 0–100 %
Датчик 2	NTC — температура, −20…80 °C
Датчик 3	DHT22 — влажность, 0–100 %
Исполнительное устройство	Реле — вытяжка
Светодиод-индикатор	Жёлтый, заданный ток 15 мА
Период опроса T	1 с
Порог события	Газ s1 > 40 %

Содержание курсового проекта

Введение

1. Постановка задачи (вариант №17)
2. Структура и алгоритм работы системы
3. Разработка аппаратной части
4. Разработка прошивки
5. Проектирование базы данных
6. Загрузка и контроль данных в Node-RED
7. Тестирование и результаты
8. Расчёт объёма данных
9. Управление версиями Git
10. Предложения по модернизации

Выводы; список сокращений; список литературы

ПРИЛОЖЕНИЯ

А. Структурная схема
Б. Схема электрическая принципиальная
В. Перечень элементов
Г. Модель в Wokwi
Д. Листинг программы Arduino
Е. Листинг функции разбора журнала Node-RED
Ж. Скрипты базы данных
И. Журнал Serial и загрузочный скрипт
К. История Git

Список рекомендованной литературы

Петин В.А. Arduino и Raspberry Pi в проектах Internet of Things. — СПб.: БХВ-Петербург, 2016.
Чакон С., Штрауб Б. Pro Git. — 2-е изд. — https://git-scm.com/book/ru/v2.
ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования к выполнению.
Документация SQLite. — https://www.sqlite.org/docs.html.

Руководитель проекта _____ (Борисов Е.Н.)

Студент _____ (Напалков Г.В.)

Содержание

Введение 5
1. Постановка задачи 7
2. Структура и алгоритм работы системы 9
2.1. Структурная схема 9
2.2. Алгоритм функционирования 9
2.3. Протокол обмена данными 11
3. Разработка аппаратной части 12
3.1. Выбор компонентов и подключение 12
3.2. Расчёт резистора и оценка питания 13
3.3. Принципиальная схема в EasyEDA 14
3.4. Модель в Wokwi 14
3.5. Расчёт шага АЦП 15
4. Разработка прошивки 16
5. Проектирование базы данных 17
5.1. Модель данных 17
5.2. Описание таблиц 17
5.3. Контрольные запросы 18
6. Загрузка и контроль данных в Node-RED 19
6.1. Состав потока 19
6.2. Функция разбора журнала 20
6.3. Запуск 20
7. Тестирование и результаты 21
7.1. Штатный сценарий 21
7.2. Сценарий неисправности датчика 23
7.3. Повторная загрузка 24
7.4. Сводка результатов 24
8. Расчёт объёма данных 25
9. Управление версиями Git 26
10. Предложения по модернизации 27
Выводы 28
Список используемых сокращений 29
Список используемой литературы 30
Приложение А. Структурная схема 31
Приложение Б. Схема электрическая принципиальная 32
Приложение В. Перечень элементов 33
Приложение Г. Модель в Wokwi 34
Приложение Д. Листинг программы Arduino 35
Приложение Е. Листинг функции Node-RED 36
Приложение Ж. Скрипты базы данных 38
Приложение И. Журнал Serial и загрузочный скрипт 39
Приложение К. История Git 40

Введение

Интеллектуальные интегрированные системы объединяют датчики, микроконтроллеры, исполнительные устройства и программные средства обработки данных. Сопровождение такой системы предполагает не только получение показаний, но и проверку их достоверности, регистрацию событий и возможность воспроизвести результат по сохранённому журналу.

Для кухни ресторана важны контроль загазованности и наблюдение за параметрами микроклимата. В учебной модели датчик MQ2 задаёт относительный уровень газа, терморезистор NTC позволяет оценивать температуру, а DHT22 — относительную влажность. При превышении установленного порога контроллер выдаёт команду на включение вытяжки и световой индикации.

До изготовления устройства его структуру и программное обеспечение удобно проверить виртуально. В проекте используются EasyEDA для принципиальной схемы, Wokwi для модели Arduino Uno, Node-RED для обработки текстового журнала и SQLite для хранения результатов. Файлы и изменения проекта учитываются в Git [1–6].

Объектом проекта является виртуальная система сбора и учёта данных на Arduino Uno для сценария «Кухня ресторана». Предметом проекта являются схемотехнические решения, прошивка, протокол обмена, структура базы данных и механизм загрузки показаний.

Цель проекта — разработать и документировать виртуальную систему варианта №17, которая получает показания трёх датчиков, выделяет превышение порога газа, сохраняет данные и обеспечивает их последующий контроль средствами SQL.

Для достижения поставленной цели необходимо решить следующие задачи:

– сформулировать требования по варианту №17 и определить диапазоны контролируемых величин;

– разработать структурную схему и алгоритм формирования событий;

– выбрать компоненты, определить подключения к Arduino Uno и рассчитать токоограничивающий резистор;

– оформить принципиальную схему и перечень элементов, подготовить модель в Wokwi;

– разработать программу опроса MQ2, NTC и DHT22 и текстовый протокол передачи данных;

– спроектировать таблицы SQLite и три контрольных запроса;

– настроить поток Node-RED для проверки журнала и повторяемой загрузки в базу;

– проверить штатный журнал, выход значения за диапазон и повторную загрузку;

– оценить объём данных и представить фактическую историю Git.

Записка подготовлена по исходным файлам проекта. Численные результаты в разделе 7 получены повторным выполнением приложенной функции разбора и SQL-скриптов над предоставленными журналами. Проверка включает разбор сообщений, формирование SQL и контроль записей в базе данных.

Проект представляет учебную виртуальную модель. Уровень газа 0–100 % выражается в относительной шкале АЦП. Реле моделирует канал управления вытяжкой, а программная часть обеспечивает регистрацию показаний и событий.

1. Постановка задачи

По варианту №17 разрабатывается система контроля кухни ресторана. Аналоговый выход MQ2 используется для определения условного уровня газа. Датчик NTC служит источником температуры, DHT22 — влажности. Реле K1 представляет канал управления вытяжкой, жёлтый светодиод показывает состояние тревоги.

Таблица 1 — Входные данные варианта №17

Параметр	Значение
Сценарий	Кухня ресторана
Датчик 1 (аналоговый)	MQ2 — газ, 0–100 %
Датчик 2	NTC — температура, −20…80 °C
Датчик 3	DHT22 — влажность, 0–100 %
Исполнительное устройство	Реле — вытяжка
Светодиод-индикатор	Жёлтый, заданный ток 15 мА
Период опроса T	1 с
Порог события	Газ s1 > 40 %

Система должна выполнять следующие функции:

1) формировать строки DATA с номером измерения и показаниями трёх датчиков с номинальным периодом T = 1 с;

2) при s1 > 40 % включать реле и жёлтый светодиод, передавать EVENT типа THRESHOLD; при s1 ≤ 40 % выключать выходы и передавать NORMAL;

3) формировать событие только при изменении состояния, не повторяя его в каждом цикле;

4) сохранять строки Serial Monitor в текстовый файл data/log.txt;

5) автоматически загружать текущий журнал в таблицы readings и events базы SQLite;

6) при конечном числовом значении за допустимым диапазоном формировать FAULT с указанием датчика;

7) отклонять некорректный журнал при отсутствии обязательного значения, неверном номере или нечисловом показании, не заменяя текущие данные базы;

8) выполнять три запроса: десять последних показаний, минимум/среднее/максимум по датчикам, список событий.

Допустимые диапазоны для контроля данных: s1 — 0–100 %, s2 — −20…80 °C, s3 — 0–100 %. Выход за диапазон и превышение технологического порога — разные ситуации: значение газа 50 % допустимо для шкалы датчика, но вызывает тревогу; значение 999 % недопустимо и фиксируется как FAULT.

В используемой реализации файл журнала переносится из Wokwi вручную. Автоматизирована следующая часть цепочки: обнаружение сохранения файла, разбор строк, подготовка SQL и загрузка в SQLite. Сохранённый журнал служит входом автоматизированной программной обработки.

Результаты проекта: принципиальная схема, перечень элементов, модель Wokwi, скетч variant_17.ino, схема базы данных и запросы, поток Node-RED, журналы проверок и репозиторий Git. Полные листинги и изображения приведены в приложениях.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, Arduino Uno, реле и светодиода, а также программной цепочки сохранения данных (рисунок 1). MQ2 подключён к A0, NTC — к A1, DHT22 — к D2. Выходы D7 и D8 управляют реле и индикацией.

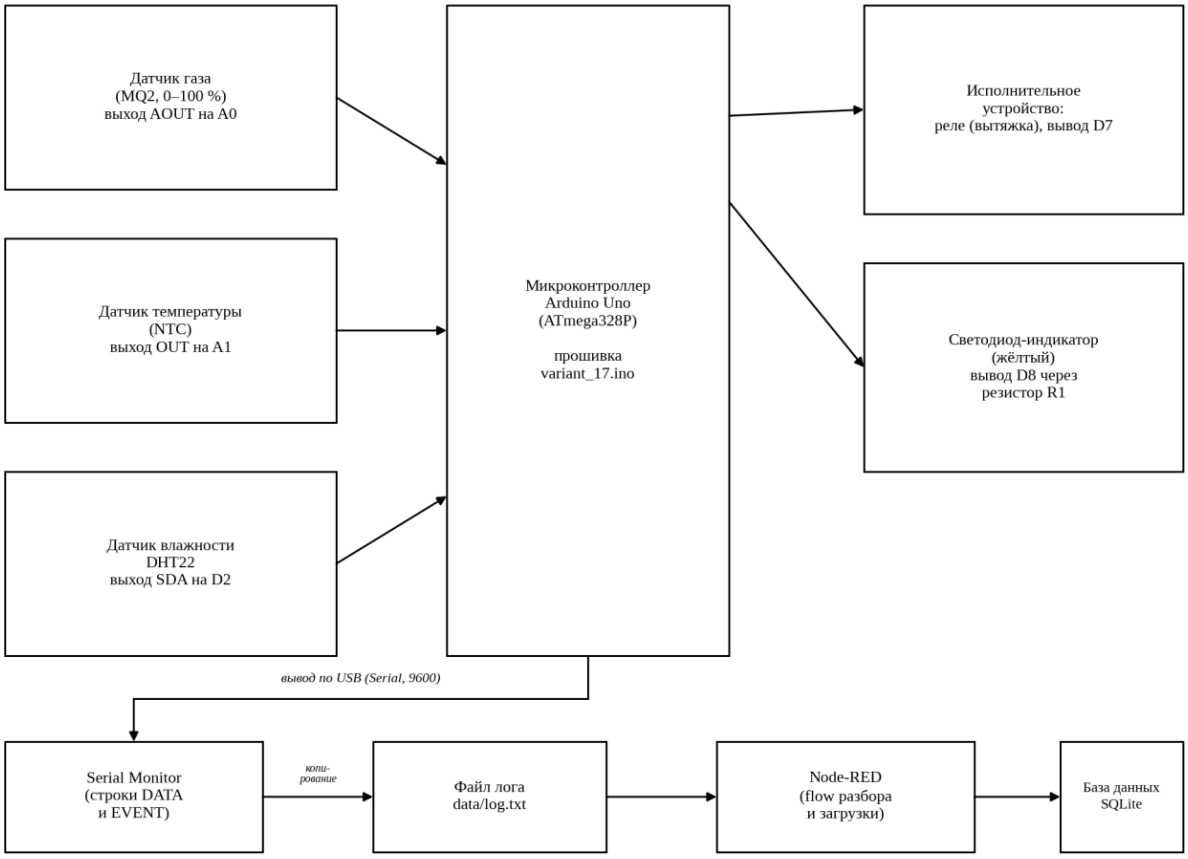


Рисунок 1 — Структурная схема системы

Текст Serial Monitor копируется в data/log.txt. Node-RED считывает файл и передаёт подготовленный SQL программе sqlite3. Увеличенная структурная схема приведена в приложении А.

2.2. Алгоритм функционирования

После запуска настраиваются порт, библиотека DHT и выходы. Номер измерения n равен нулю, alarm сброшен. В каждом цикле n увеличивается, считываются s1, s2, s3 и передаётся строка DATA.

Условие тревоги определяется только уровнем газа: s1 > 40 %. При переходе в тревогу устанавливается alarm, включаются выходы и передаётся THRESHOLD. При возврате к s1 ≤ 40 % состояние сбрасывается и передаётся NORMAL. В конце цикла выполняется delay(1000).

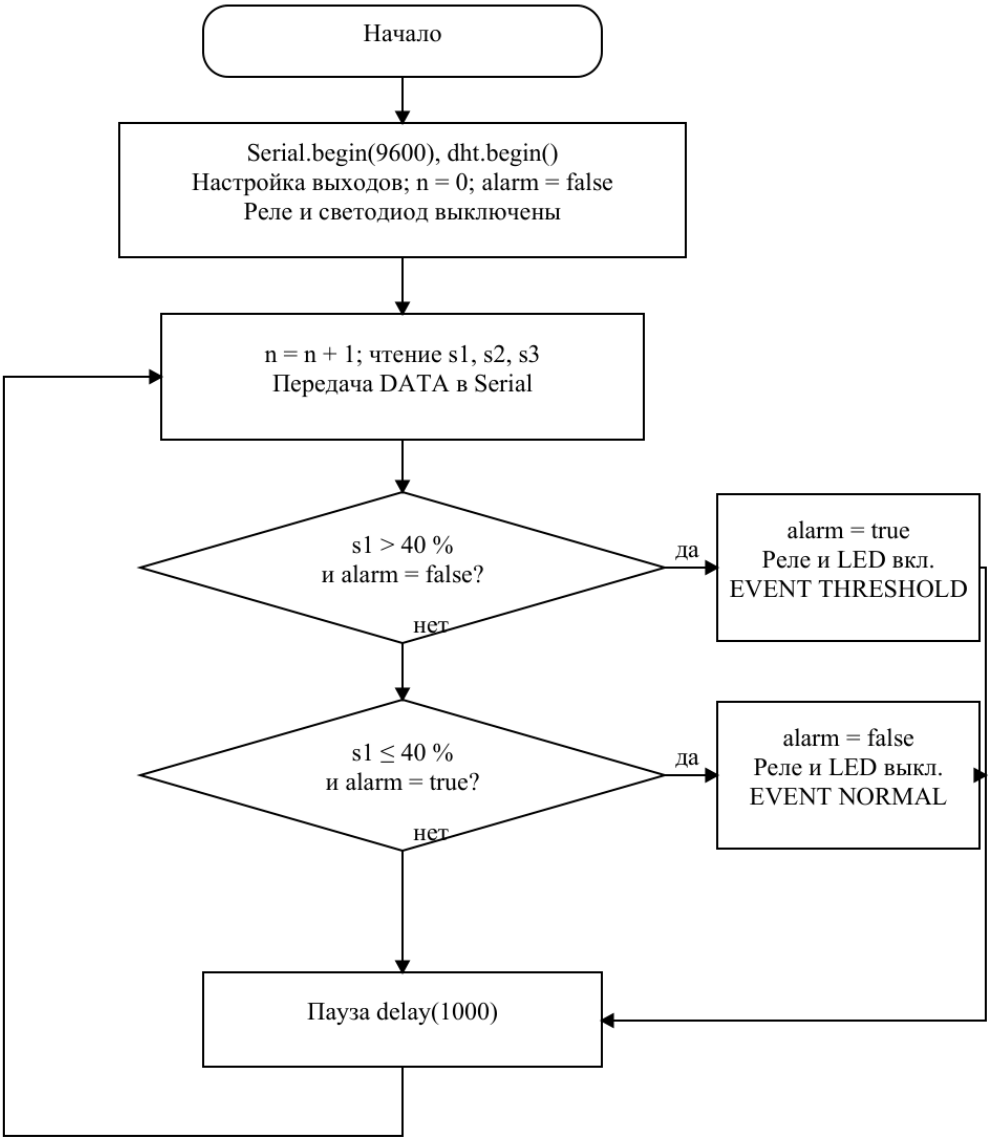


Рисунок 2 — Алгоритм программы Arduino варианта №17

2.3. Протокол обмена данными

Для обмена используются текстовые строки с парами «ключ=значение». Скорость Serial составляет 9600 бод. Каждая строка заканчивается переводом строки; десятичный разделитель в журнале — точка. Формат удобен для ручного просмотра и разбора в Node-RED.

Таблица 2 — Формат строк последовательного порта

Тип	Формат и пример
DATA	DATA n=<номер> s1=<газ> s2=<температура> s3=<влажность> DATA n=12 s1=66.57 s2=24.0 s3=50.0
EVENT	EVENT n=<номер> type=THRESHOLD | NORMAL EVENT n=12 type=THRESHOLD

Газ выводится с двумя десятичными знаками, температура и влажность — с одним. При ошибке чтения влажности скетч выводит s3=nan. Такая строка не является допустимым числовым DATA для текущего обработчика: Node-RED прекращает загрузку и сообщает об ошибке.

Номер n связывает показания и событие одного цикла. В базе время вычисляется по формуле $t_s = n \cdot T$, где $T = 1$ с. Таким образом формируется относительная временная шкала по номеру измерения. Первый опрос выполняется сразу после setup(), а длительность чтения датчиков добавляется к паузе delay().

Событие FAULT формирует Node-RED при выходе конечного числового значения за границы диапазона. Например, s1=999 сохраняется в readings для анализа исходных данных и дополнительно создаёт событие FAULT.

При каждом запуске скетча нумерация n начинается заново. Каждый журнал обрабатывается как самостоятельный сеанс, а база содержит показания и события текущего файла.

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

В модели используются Arduino Uno на ATmega328P, модуль MQ2, модуль NTC, DHT22, одноканальный модуль реле и жёлтый светодиод с резистором 200 Ом. Подключения соответствуют скетчу и последней приложенной принципиальной схеме.

Таблица 3 — Подключение компонентов к Arduino Uno

Сигнал	Компонент	Вывод	Режим
Уровень газа s1	B1, MQ2, SIG/AOUT	A0	Аналоговый вход
Температура s2	B2, NTC, OUT	A1	Аналоговый вход
Влажность s3	B3, DHT22, DATA	D2	Цифровой обмен
Вытяжка	K1, вход IN	D7	Выход
Индикация	LED1 через R1 = 200 Ом	D8	Выход
Питание	Модули B1–B3, K1	5 V, GND	Общий провод

АЦП Arduino Uno имеет 10 разрядов и выдаёт коды от 0 до 1023. Для MQ2 используется пересчёт G = 100·raw/1023; расчёт шага АЦП приведён в разделе 3.5 [2].

Порог 40 % соответствует коду 409,2. Условие G > 40 % выполняется начиная с кода 410: при 409 получается около 39,98 %, при 410 — около 40,08 %. Для управления используется относительная шкала газа 0–100 %.

Температура вычисляется по нелинейной BETA-модели NTC с B = 3950 K и опорной температурой 298,15 K. Эта формула связывает код АЦП с температурой через температурную зависимость сопротивления терморезистора.

3.2. Расчёт резистора и оценка питания

Для жёлтого светодиода принимаются Uпит = 5 В, Uпр = 2,0 В и заданный ток I = 15 мА. Сопротивление и мощность резистора:

$$R1 = (5 - 2{,}0) / 0{,}015 = 200 \text{ Ом}; \quad P = 0{,}015^2 \cdot 200 = 0{,}045 \text{ Вт}.$$

Выбран R1 = 200 Ом мощностью 0,25 Вт. Включённая цепь LED1–R1 потребляет от шины 5 В ток 15 мА и мощность 0,075 Вт.

Расчёт питания выполняется при включённых реле и индикаторе. Для Arduino Uno принято типовое потребление 50 мА. Для MQ2 по даташиту Рн ≤ 900 мВт: Iн = 0,9 / 5 = 0,18 А; с расчётным резервом 10 мА на сигнальную цепь и электронику модуля принято 190 мА [13].

Для DHT22 используется ток 1,5 мА из спецификации [14]. Для оценки NTC принят делитель с постоянным резистором 10 кОм и RNTC = 10 кОм при 25 °C: I = 5 / 20 000 = 0,25 мА. Предельный ток делителя — 0,5 мА; с резервом 1 мА на электронику принято 1,5 мА. Максимальная мощность на NTC в таком делителе равна $5^2 / (4 \cdot 10\,000) = 0{,}625$ мВт, что ниже допустимых 100 мВт по документу Molex [15].

Для одноканального релейного модуля K1 принят расчётный ток 80 мА: 72 мА на катушку 5 В и 8 мА на цепь управления и индикацию. Катушка питается от шины 5 В; D7 передаёт управляющий сигнал.

Таблица 4 — Расчёт потребления от шины 5 В

Потребитель	Ток, мА	Мощность, Вт	Основание
Arduino Uno	50	0,250	Типовое потребление платы
MQ2, модуль	190	0,950	180 мА нагреватель + 10 мА резерв
NTC, измерительная цепь	1,5	0,0075	0,5 мА делитель + 1 мА резерв
DHT22	1,5	0,0075	По спецификации
Реле K1, включено	80	0,400	Принято: катушка и управление
LED1 с R1	15	0,075	Расчёт при токе 15 мА
Итого	338	1,690	Все узлы включены

Суммарный ток IΣ = 338 мА; мощность PΣ = 5·0,338 = 1,69 Вт. С запасом 30 % требуется 439,4 мА. Расчёт укладывается в бюджет USB 5 В / 500 мА при доступности этого тока. Для отдельного питания выбран стабилизированный источник 5 В / 1 А. За сутки при непрерывном максимальном расчётном режиме E = 1,69·24 = 40,56 Вт·ч. Двигатель вытяжки питается по отдельной силовой цепи и учитывается отдельно.

3.3. Принципиальная схема в EasyEDA

Принципиальная схема выполнена в EasyEDA и включена в приложение Б в том виде, в котором она предоставлена для отчёта. Использованы обозначения A1 — Arduino Uno, B1 — MQ2, B2 — NTC, B3 — DHT22, K1 — реле, LED1 — жёлтый индикатор, R1 — резистор 200 Ом. Перечень элементов приведён в приложении В.

Сначала размещаются компоненты, затем соединяются питание и общий провод, после чего проводятся сигнальные цепи. Проверяются номера выводов, отсутствие замыканий питания и соответствие соединений таблице 3. Документы содержат рамку и основную надпись; правила выполнения схем рассматриваются в ЕСКД [7–9].

Индикатор LED1 подключён последовательно с внешним токоограничивающим резистором R1. В комментариях прошивки он обозначен HL1. Оба обозначения относятся к жёлтому светодиоду, управляемому выводом D8.

3.4. Модель в Wokwi

Модель содержит семь элементов: плату, три датчика, реле, светодиод и резистор. Уровень газа изменяется в настройках MQ2, температура — у NTC, влажность — у DHT22. Скриншот сохранён в docs/scheme.png и приведён в приложении Г [3].

Ссылка на модель: https://wokwi.com/projects/479217730910433281.

Подключения модели заданы в diagram.json. В прошивке команда включения реле задаётся уровнем HIGH на D7, команда выключения — уровнем LOW. Текст из Serial Monitor сохраняется пользователем в data/log.txt для последующей обработки.

3.5. Расчёт шага АЦП

Аналого-цифровой преобразователь Arduino Uno имеет разрядность n = 10, поэтому число уровней квантования составляет $N = 2^n = 2^{10} = 1024$. При принятом опорном напряжении Uоп = 5 В и диапазоне от 0 до 5 В номинальный шаг квантования равен:

$\Delta U = U_{оп} / 2^n = 5 / 1024 = 0{,}004883$ В ≈ 4,88 мВ.

При преобразовании кода в относительный уровень газа по формуле G = 100·raw/1023 соседним кодам соответствует изменение показания $\Delta G = 100/1023 \approx 0{,}0978$ процентного пункта. Делитель 1023 используется для отображения максимального кода 1023 в 100 %, а 1024 — для расчёта номинального шага квантования напряжения.

4. Разработка прошивки

Прошивка variant_17.ino написана на C++ с использованием Arduino API и библиотеки DHT. Полный исходный текст приведён в приложении Д. Номера выводов, T_MS = 1000, шкала газа и THRESHOLD = 40 вынесены в константы [2, 10].

Таблица 5 — Функции программы

Функция	Назначение
setup()	Serial 9600, запуск DHT, настройка и сброс выходов
loop()	Чтение датчиков, DATA, проверка перехода, EVENT, пауза
readS1()	Код A0 → условный уровень газа 0–100 %
readS2()	Код A1 → температура по BETA-формуле NTC
readS3()	Влажность dht.readHumidity()
actuatorOn() / actuatorOff()	Установка HIGH / LOW на D7
conditionMet()	Проверка s1 > 40 %

Переменная alarm хранит предыдущее состояние. Благодаря ей при длительном превышении порога формируется одна запись THRESHOLD, а не событие каждую секунду. Показания температуры и влажности сохраняются, но в управлении реле не участвуют.

Функция readS3() получает влажность через dht.readHumidity(). Библиотека DHT управляет обменом с датчиком и сохраняет результат между обновлениями. Цикл программы использует доступное значение влажности при формировании очередной строки DATA [10].

После передачи показаний программа сравнивает s1 с порогом 40 %. Переход в тревогу включает реле и светодиод и формирует THRESHOLD; возврат к норме выключает выходы и формирует NORMAL. Завершающая пауза delay(T_MS) равна 1000 мс. В readS2() значение raw снизу ограничивается единицей для вычисления температуры. При нечисловом результате чтения DHT в журнал выводится nan, которое распознаёт обработчик Node-RED.

5. Проектирование базы данных

5.1. Модель данных

Выбрана встраиваемая СУБД SQLite. База db/variant.db состоит из таблиц readings — показания и events — события (рисунок 3). Каждое показание отдельного датчика занимает отдельную строку. Такой «длинный» формат позволяет группировать данные по sensor [5].

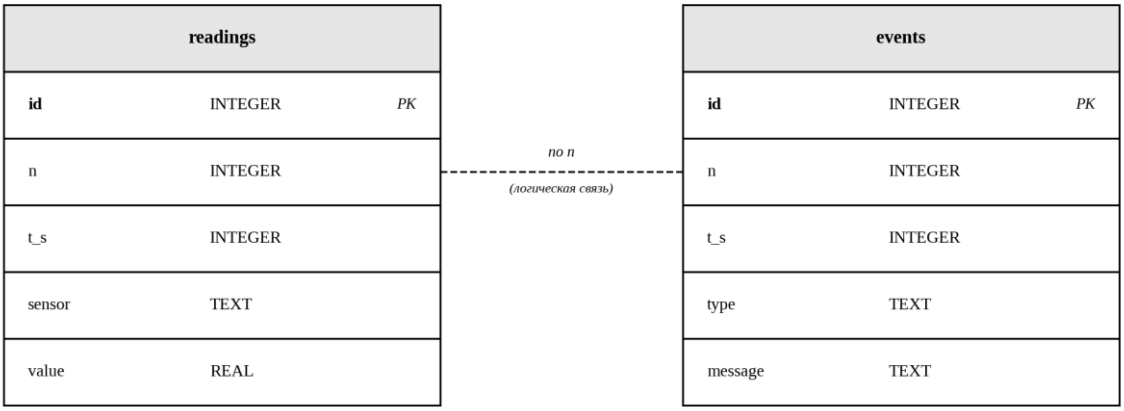


Рисунок 3 — Модель данных SQLite

Таблицы связаны логически по номеру n: каждому циклу соответствуют три строки показаний и относящиеся к нему события. Поле n позволяет совместно анализировать обе таблицы.

5.2. Описание таблиц

Таблица 6 — Структура таблицы readings

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	Уникальный идентификатор
n	INTEGER, NOT NULL	Номер цикла из журнала
t_s	INTEGER, NOT NULL	Условное время n·T, с
sensor	TEXT, NOT NULL	s1, s2 или s3
value	REAL, NOT NULL	Числовое показание

Таблица 7 — Структура таблицы events

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	Уникальный идентификатор
n	INTEGER, NOT NULL	Номер цикла события
t_s	INTEGER, NOT NULL	Условное время, с
type	TEXT, NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	Пояснение; для FAULT — имя датчика

Скрипт schema.sql использует CREATE TABLE IF NOT EXISTS и допускает повторный запуск. Значения вне диапазона не запрещены ограничениями CHECK: они сохраняются как исходные данные, а нарушение отмечается в events. Это позволяет видеть причину неисправности, а не терять ошибочное показание.

5.3. Контрольные запросы

Первый запрос сортирует readings по id по убыванию и выводит 10 строк. Поскольку в одном цикле создаются три записи, это десять показаний, а не десять полных циклов.

Второй запрос группирует строки по sensor и вычисляет MIN(value), ROUND(AVG(value), 2), MAX(value). Средние значения разных физических величин между собой не объединяются. Третий запрос выводит все события в порядке id. Тексты запросов приведены в приложении Ж.

При анализе реальных данных FAULT-показания необходимо исключать из расчёта нормального режима или выводить отдельной выборкой: базовый запрос AVG учитывает в том числе значение 999. Для учебного теста штатный и аварийный журналы загружаются раздельно.

6. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Поток импортируется из nodered/flow.json. Файл читается целиком, функция формирует SQL-скрипт, затем узел file записывает db/load.sql и передаёт сообщение узлу exec. Последовательное соединение узлов обеспечивает запуск sqlite3 после записи файла [4].

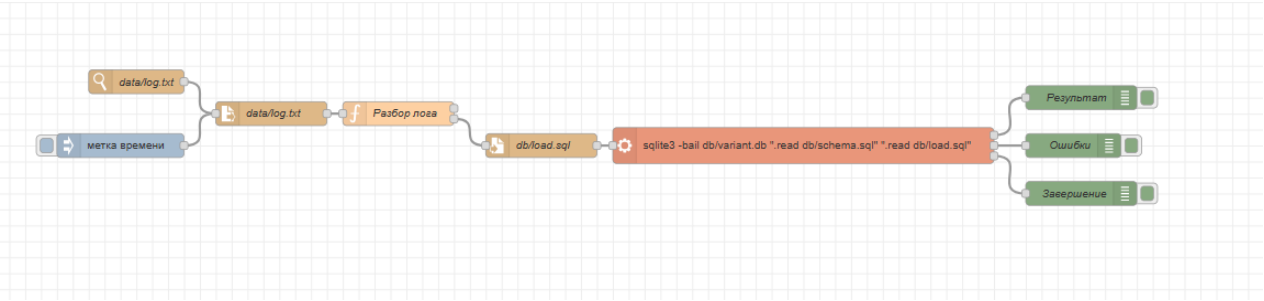


Рисунок 4 — Предоставленный поток Node-RED

Таблица 8 — Узлы и настройки потока

Узел / тип	Назначение
watch	Следить за изменениями data/log.txt
inject	Ручной запуск загрузки
file in	Прочитать data/log.txt целиком, UTF-8
function, два выхода	Проверить строки; SQL-скрипт на втором выходе
file	Перезаписать db/load.sql
exec	Запустить sqlite3 -bail, прочитать schema.sql и load.sql
debug (три узла)	Показать stdout, stderr и код завершения

В узле exec отключено добавление msg.payload к командной строке. Первый выход function не используется: экспортированный обработчик возвращает [null, {payload: SQL}]. Для наблюдения результата загрузки контролируются ошибки и код завершения sqlite3.

6.2. Функция разбора журнала

Функция удаляет BOM, разбивает текст на строки и выделяет тип сообщения и поля. Служебные строки пропускаются. Номер n должен быть положительным безопасным целым числом. Для DATA обязательны s1, s2, s3; каждое значение преобразуется Number() и проверяется Number.isFinite().

Конечное числовое показание всегда записывается в readings. При выходе за диапазон добавляется FAULT. Для EVENT допускаются только THRESHOLD и NORMAL. Отсутствующее поле, nan, неверный номер или неизвестный тип события приводят к node.error() и return null. До узла записи SQL сообщение в этом случае не доходит, поэтому текущая база не заменяется.

Скрипт начинается с BEGIN, очистки обеих таблиц и сброса их счётчиков sqlite_sequence; заканчивается COMMIT. Таким образом, база представляет снимок всего текущего журнала, а не накопительную историю разных файлов. Повторная загрузка не увеличивает число записей. Транзакция и ключ -bail предотвращают фиксацию частично выполненной загрузки при ошибке SQL [5].

6.3. Запуск

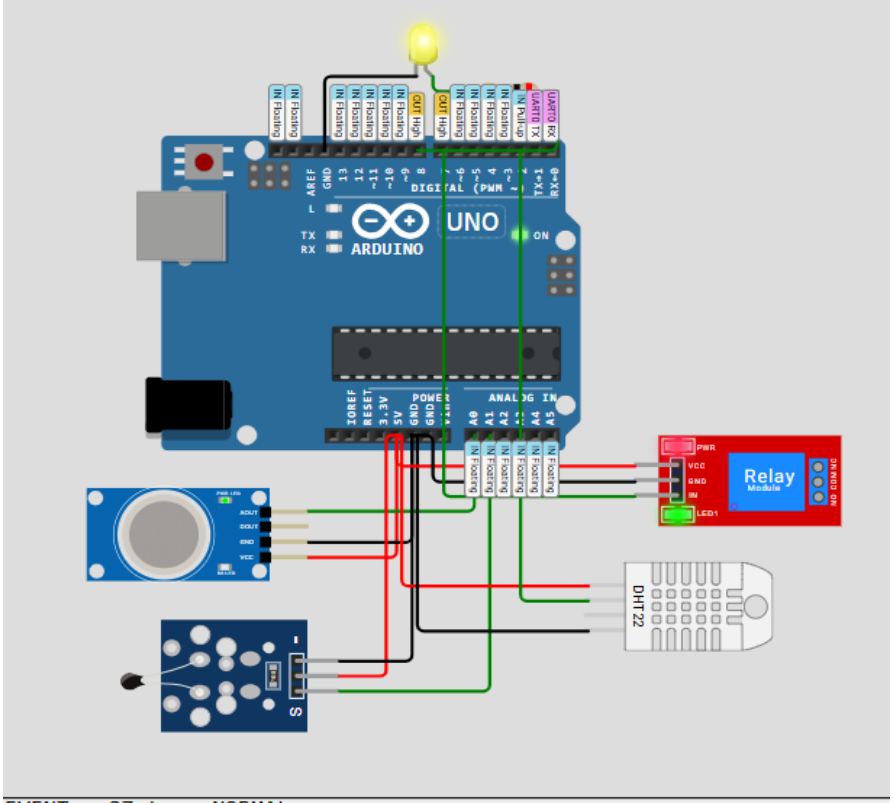
На Windows выполняется start-nodered.bat. Скрипт добавляет портативные Node.js и SQLite из %USERPROFILE%\tools в PATH и переходит в папку проекта. Затем в браузере открывается http://localhost:1880, импортируется flow.json и выполняется развёртывание потока.

После сохранения data/log.txt загрузка запускается через watch либо вручную через inject. Проверка базы из папки проекта:

```
sqlite3 -header -column db/variant.db ".read db/queries.sql"
```

Рабочая папка проекта является основой относительных путей data/log.txt, db/schema.sql, db/load.sql и db/variant.db. Командный файл задаёт окружение запуска, а узлы debug отображают результат выполнения загрузки.

7. Тестирование и результаты




```
EVENT n=87 type=NORMAL
DATA n=88 s1=25.90 s2=24.0 s3=53.0
DATA n=89 s1=25.90 s2=24.0 s3=53.0
DATA n=90 s1=96.09 s2=24.0 s3=53.0
EVENT n=90 type=THRESHOLD
DATA n=91 s1=96.09 s2=24.0 s3=53.0
DATA n=92 s1=96.09 s2=24.0 s3=53.0
```

Рисунок 5 – работающая симуляция Wokwi

7.1. Штатный сценарий

Для проверки использован предоставленный data/log.txt: 10 строк DATA с номерами n = 9…18 и 4 строки EVENT. Рассматриваемый фрагмент начинается при n = 9. События фиксируют изменения состояния внутри этого фрагмента при n = 10, 12, 16 и 17.

Таблица 9 — Штатный журнал варианта №17

n	t_s, c	s1, %	s2, °C	s3, %	Событие
9	9	47,90	24,0	50,0	—
10	10	27,08	24,0	50,0	NORMAL
11	11	27,08	24,0	50,0	—
12	12	66,57	24,0	50,0	THRESHOLD
13	13	62,66	24,0	50,0	—
14	14	50,73	24,0	50,0	—

n	t_s, с	s1, %	s2, °C	s3, %	Событие
15	15	72,92	24,0	50,0	—
16	16	30,50	24,0	50,0	NORMAL
17	17	83,97	24,0	50,0	THRESHOLD
18	18	83,97	24,0	50,0	—

Исходная функция из flow.json выполнена над этим журналом; полученный SQL исполнен в отдельной тестовой базе SQLite с исходной schema.sql. Результат: 30 показаний и 4 события, FAULT нет. Исходные файлы проекта при проверке не изменялись.

Таблица 10 — Запрос 1: десять последних показаний

id	n	t_s	sensor	value
30	18	18	s3	50.0
29	18	18	s2	24.0
28	18	18	s1	83.97
27	17	17	s3	50.0
26	17	17	s2	24.0
25	17	17	s1	83.97
24	16	16	s3	50.0
23	16	16	s2	24.0
22	16	16	s1	30.5
21	15	15	s3	50.0

Таблица 11 — Запрос 2: минимум, среднее и максимум

sensor	min_v	avg_v	max_v
s1	27,08	55,34	83,97
s2	24,0	24,0	24,0
s3	50,0	50,0	50,0

Газ изменялся от 27,08 до 83,97 % учебной шкалы. Температура в данном фрагменте постоянна — 24,0 °C, влажность — 50,0 %. Эти значения характеризуют температурно-влажностные условия рассматриваемого сценария.

Таблица 12 — Запрос 3: события штатного журнала

id	n	t_s, c	type
1	10	10	NORMAL
2	12	12	THRESHOLD
3	16	16	NORMAL
4	17	17	THRESHOLD

События совпадают с переходами через порог: при n = 12 и 17 газ становится выше 40 %, при n = 10 и 16 возвращается к норме. Показания выше 40 % между переходами не создают повторных EVENT.

7.2. Сценарий неисправности датчика

Для аварийного теста использован файл data/log_fault.txt:

```
DATA n=1 s1=999 s2=25.0 s3=50.0
```

Повторное выполнение обработчика и SQL даёт 3 записи readings и одну запись events: n = 1, t_s = 1, type = FAULT, message = «датчик s1 вне диапазона». Ошибочное значение 999 сохраняется; температура 25 °C и влажность 50 % допустимы. Такая же аварийная выборка находится в предоставленной db/variant.db.

7.3. Повторная загрузка

SQL штатного журнала выполнен повторно в той же тестовой базе. После каждой загрузки остаются 30 строк readings и 4 строки events, идентификаторы начинаются заново. Проверка подтверждает повторяемость загрузки снимка текущего журнала.

Дополнительно проверены неполная строка DATA, значение s3=nan и пустой файл. Во всех случаях исходная функция вернула null и сообщение об ошибке, не выдав SQL на запись. Таким образом, эти ошибки отличаются от числового выхода за диапазон, который создаёт FAULT.

7.4. Сводка результатов

Таблица 13 — Результаты программных проверок

Проверка	Ожидалось	Получено
Штатный журнал, 10 циклов	30 показаний, 4 события	30 и 4; FAULT нет
Превышение порога	THRESHOLD при n = 12, 17	Соответствует
Возврат к норме	NORMAL при n = 10, 16	Соответствует
s1 = 999	3 показания и 1 FAULT	Соответствует
Повторная загрузка	Число строк не растёт	30 и 4
Нет s3; s3=nan; пусто	Отклонение без SQL	return null, сообщение ошибки

Проверка охватывает разбор сообщений, формирование SQL, регистрацию показаний и событий, а также повторную загрузку. Штатный журнал подтверждает соответствие событий переходам через порог 40 %. Аварийный журнал демонстрирует регистрацию FAULT при выходе s1 за допустимый диапазон.

Состав и соединения виртуальной модели представлены на скриншоте Wokwi. Совокупность схемы, прошивки, журналов и SQL-запросов описывает полный путь данных от датчиков до сохранённых результатов.

8. Расчёт объёма данных

Оценка выполнена для условного непрерывного накопления данных при T = 1 с. Один цикл формирует три строки readings. Для предварительного расчёта принято 40 Б на запись; фактический размер SQLite зависит от представления чисел, страниц и индексов.

Таблица 14 — Оценка объёма данных

Показатель	Расчёт	Результат
Циклов в сутки	86 400 / 1	86 400
Строк readings в сутки	86 400 × 3	259 200
Размер строки, оценка	id, n, t_s, sensor, value	≈ 40 Б
Объём readings за сутки	259 200 × 40	≈ 10,37 МБ
За 30 суток	10 368 000 × 30	≈ 311,04 МБ
За год (365 суток)	10 368 000 × 365	≈ 3,78 ГБ
Пример DATA с CRLF	DATA n=86400 s1=50.00 s2=24.0 s3=50.0	39 Б
DATA-журнал за сутки	86 400 × 39	≈ 3,37 МБ

Используются десятичные единицы: 1 МБ = 10^6 Б, 1 ГБ = 10^9 Б. События, индексы и служебные структуры в оценку не включены. Длина DATA изменяется с числом разрядов n и значений; 39 Б — размер приведённой строки, а не постоянная длина протокола.

Годовая оценка характеризует объём непрерывно накапливаемых показаний. В реализованном режиме база хранит снимок текущего журнала: при загрузке содержимое таблиц заменяется записями всего файла. Поэтому размер текущей выборки определяется объёмом загружаемого журнала.

Архивирование исходных журналов и резервное копирование базы обеспечивают сохранность данных. Политика хранения определяет срок доступности записей; итоговый размер файла SQLite зависит от объёма выборки, страниц базы и индексов.

9. Управление версиями Git

Ссылка на репозиторий: Usablessme/napalkov

Проект хранится в репозитории Git. В архиве присутствует каталог .git, поэтому история ниже взята из фактического журнала коммитов, а не составлена как пример. Исходники разделены на firmware, sim, db, nodered, data и docs; инструкция запуска находится в README.md [6].

```
git status
git log --oneline
```

Для командного проекта рекомендуются отдельные коммиты по законченным этапам. Перед фиксацией проверяют, что в индекс не попали временные файлы, локальные настройки и сгенерированная база. История коммитов позволяет проследить последовательность разработки и восстановить сохранённые редакции файлов.

Таблица 15 — Основные этапы фактической истории Git

Хеш	Сообщение коммита
27b9574	Добавлена схема в Wokwi (вариант 17)
bc1a5ff	Добавлена прошивка variant_17.ino
8635051	Тестовый файл неисправности log_fault.txt
ed510a1	Схема БД schema.sql и контрольные запросы
3eff7a8	Структура проекта, README, .gitignore
dc0a261	Wokwi: ссылка в README, скриншот схемы
bb295b3	ER-диаграмма и контрольные запросы
7105b6a	Схема EasyEDA, BOM и структурная схема
7162fa9	Настроена загрузка данных из лога через Node-RED в SQLite
dd5beeb	Проверено обнаружение неисправности датчика s1
a12eb6d	Заполнено описание проекта и инструкция запуска
d953205	Правки описания проекта

Полный вывод истории по убыванию времени приведён в приложении К. Файлы схемы и перечня, приложенные отдельно к заданию на отчёт, использованы в приложениях Б и В как актуальная редакция документов.

10. Предложения по модернизации

Для повышения устойчивости работы системы целесообразно добавить гистерезис управления реле и диагностику неисправностей датчиков, включая проверку предельных кодов АЦП и ошибок DHT22. Для длительного мониторинга можно реализовать автоматический сбор данных с отметкой времени, накопление записей и резервное копирование базы; применение на реальной кухне потребует отдельно проверенного оборудования газовой безопасности.

Выводы

В курсовом проекте рассмотрена виртуальная система сбора и учёта данных для кухни ресторана, вариант №17. Определены подключения MQ2, NTC и DHT22 к Arduino Uno, условие тревоги s1 > 40 % и текстовый протокол DATA/EVENT.

Оформлены структурная схема, алгоритм и аппаратные документы. Для жёлтого индикатора при Uпр = 2,0 В и токе 15 мА рассчитан резистор R1 = 200 Ом с рассеиваемой мощностью 0,045 Вт. Описание прошивки соответствует приложенному исходному коду, в том числе нелинейному пересчёту NTC и особенностям обновления DHT22.

База SQLite состоит из таблиц readings и events. Поток Node-RED проверяет обязательные поля, отмечает числовые выходы за диапазон событием FAULT и заменяет содержимое базы снимком текущего журнала. Подготовлены три контрольных запроса.

Программное воспроизведение штатного журнала подтвердило 30 показаний и 4 события; аварийного — 3 показания и 1 FAULT. Повторная загрузка не увеличивает число строк. Расчёт для возможного накопительного режима даёт около 10,37 МБ показаний в сутки без учёта событий и служебных структур.

Расчёт питания при включённых реле и индикаторе дал суммарный ток 338 мА и мощность 1,69 Вт. С учётом 30-процентного запаса требуется 439,4 мА; для отдельного питания выбран источник 5 В / 1 А.

Виртуальная модель объединяет измерительную часть, управление индикацией и реле, передачу сообщений и хранение данных. Подготовленная документация описывает состав системы, порядок запуска и воспроизводимые результаты обработки журналов.

Список используемых сокращений

Сокращение	Расшифровка
АЦП	Аналого-цифровой преобразователь
БД	База данных
ГОСТ	Государственный стандарт
ЕСКД	Единая система конструкторской документации
ИИС	Интеллектуальные интегрированные системы
СУБД	Система управления базами данных
DHT	Семейство цифровых датчиков температуры и влажности
ERC	Проверка электрических правил (Electrical Rules Check)
LED	Светоизлучающий диод (Light-Emitting Diode)
MQ2	Газочувствительный датчик с нагревателем
NTC	Терморезистор с отрицательным температурным коэффициентом
SQL	Язык структурированных запросов
UART	Универсальный асинхронный приёмопередатчик
USB	Универсальная последовательная шина
NaN	Нечисловое значение (Not a Number)
ppm	Число частей на миллион

Список используемой литературы

1. Петин В.А. Arduino и Raspberry Pi в проектах Internet of Things. — СПб.: БХВ-Петербург, 2016. — 320 с.

2. Arduino. Language Reference; Arduino Uno Rev3. — URL: https://docs.arduino.cc/language-reference/ ; https://docs.arduino.cc/hardware/uno-rev3/.

3. Wokwi. Documentation. — URL: https://docs.wokwi.com/.

4. Node-RED. Documentation. — URL: https://nodered.org/docs/.

5. SQLite. Documentation; Command Line Shell. — URL: https://www.sqlite.org/docs.html ; https://www.sqlite.org/cli.html.

6. Чакон С., Штрауб Б. Pro Git. — 2-е изд. — URL: https://git-scm.com/book/ru/v2.

7. ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования к выполнению.

8. ГОСТ 2.104-2006. ЕСКД. Основные надписи.

9. ГОСТ 2.710-81. ЕСКД. Обозначения буквенно-цифровые в электрических схемах.

10. Adafruit. DHT sensor library. Исходный код и описание библиотеки. — URL: https://github.com/adafruit/DHT-sensor-library.

11. Microchip Technology. ATmega328P. Документация микроконтроллера. — URL: https://www.microchip.com/en-us/product/ATmega328P.

12. EasyEDA. Documentation. — URL: https://docs.easyeda.com/.

13. ZYMQ-2. Технические параметры газочувствительного элемента. — Даташит DOC001184453.pdf. — 2 с.

14. iArduino. Цифровой датчик температуры и влажности DHT11, DHT22 (Trema-модуль). — Техническое описание DOC037836294.pdf. — 9 с.

15. Molex. NTC Ring Terminal Sensor Cable Assemblies. — Order No. 987652-1521. — 2019. — DOC011154357.pdf. — 2 с.

Приложение А. Структурная схема

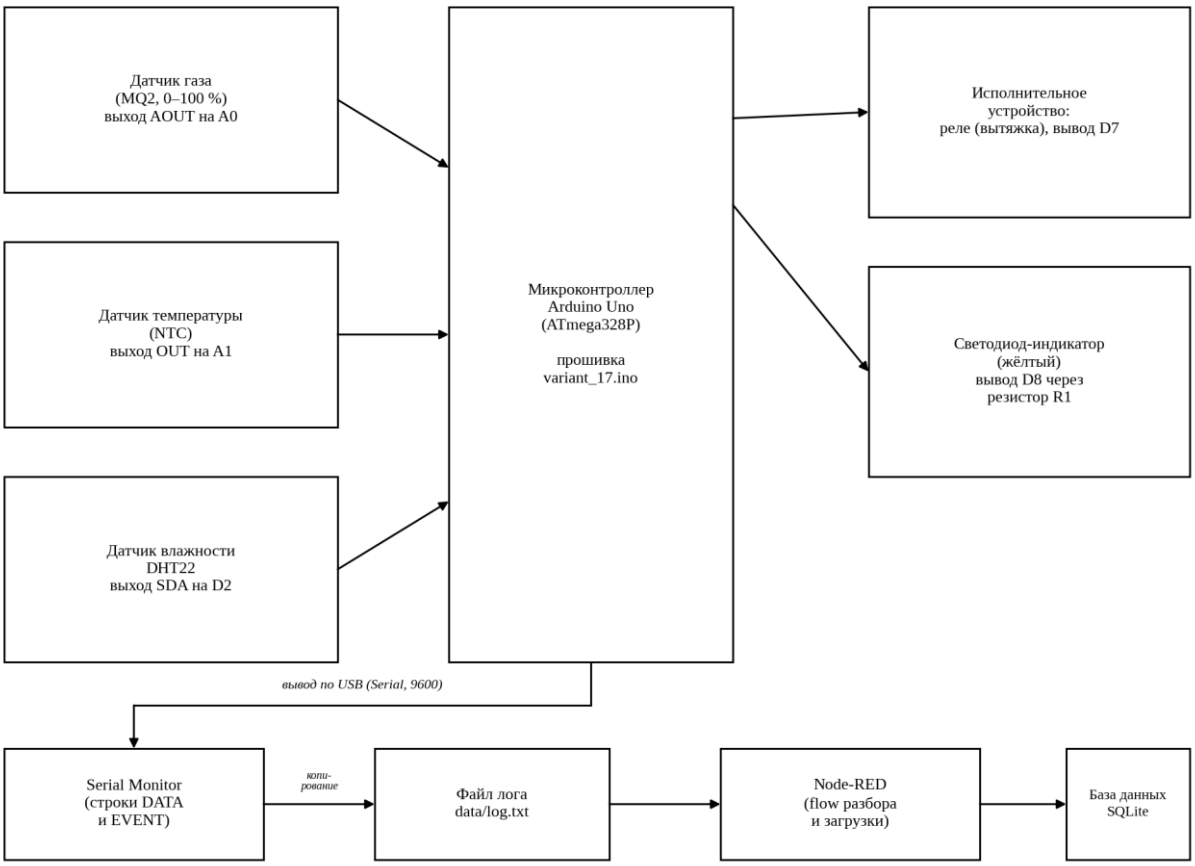
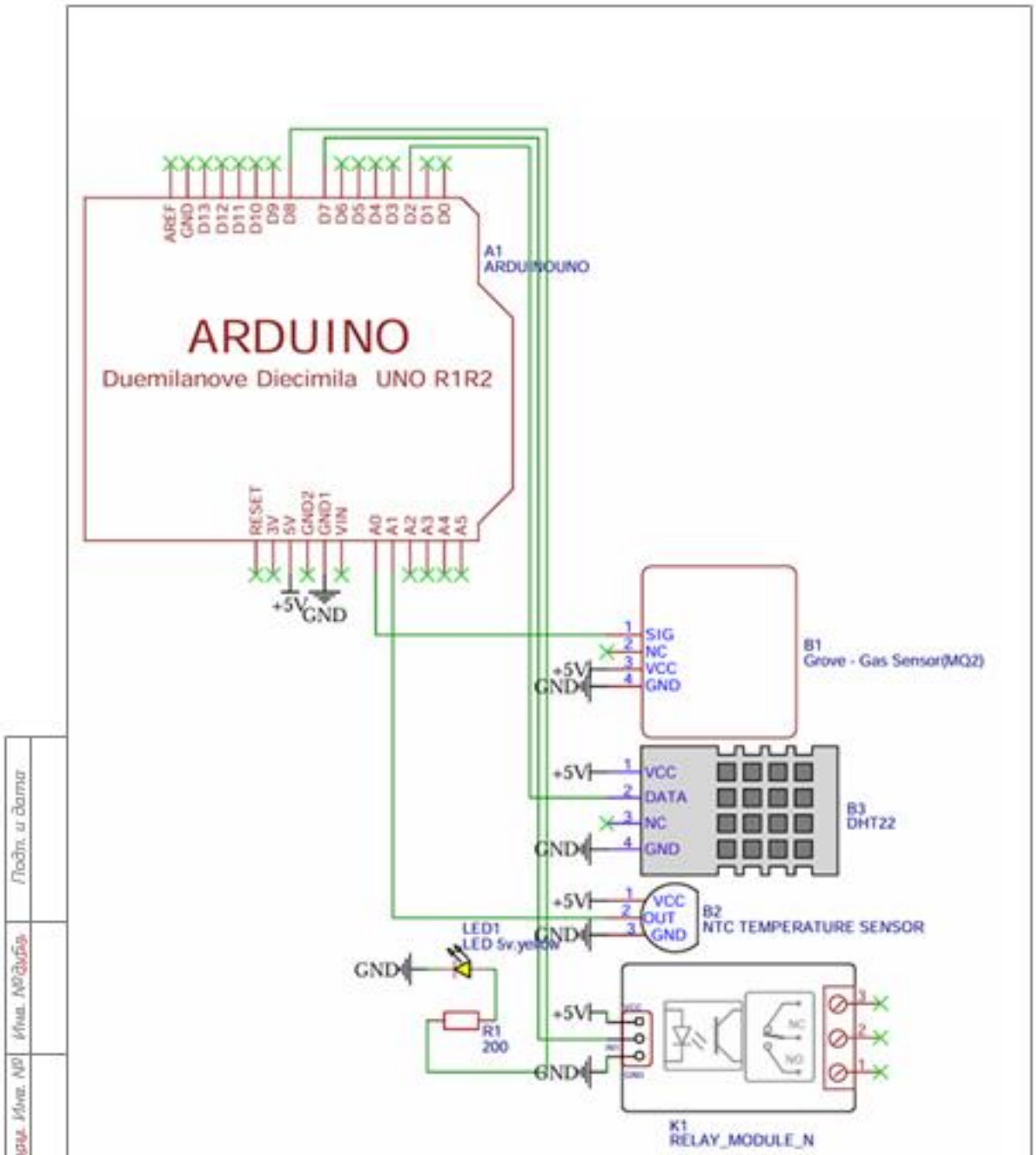


Рисунок А.1 — Структурная схема системы кухни ресторана

Обозначения интерфейсов и выводов соответствуют firmware/variant_17.ino. Перенос текста из Serial Monitor в журнал показан как отдельная ручная операция.

Приложение Б. Схема электрическая принципиальная



Приложение В. Перечень элементов

Форм	Зона	Поз.	Обозначение	Наименование	Кол.	Прим.
				Модули и платы		
			A1	Плата Arduino Uno	1	
			B1	Датчик газа MQ2	1	
			B2	Датчик температуры NTC	1	
			B3	Датчик влажности DHT22	1	
			K1	Модуль реле KY-019	1	
			LED1	Светодиод LED5v желтый	1	
			R1	Резистор 200Ом	1	

Изм.	Лист	№ докум.	Подп.	Дата	КП.5415.17.ЭЗ ПЭЗ		
Разраб.		Напалков Г.В.		01.10.25	Кухня ресторана	Лит. У	Лист 2 / Листов 2
Пров.		Барихов Е.Н.		01.10.25			
Н.контр.		Пахадина Г.А.		05.11.25	Перечень элементов	СПб ГБПОУ	
Утв.		Бычков А.Ю.		05.11.25			

Приложение Г. Модель в Wokwi

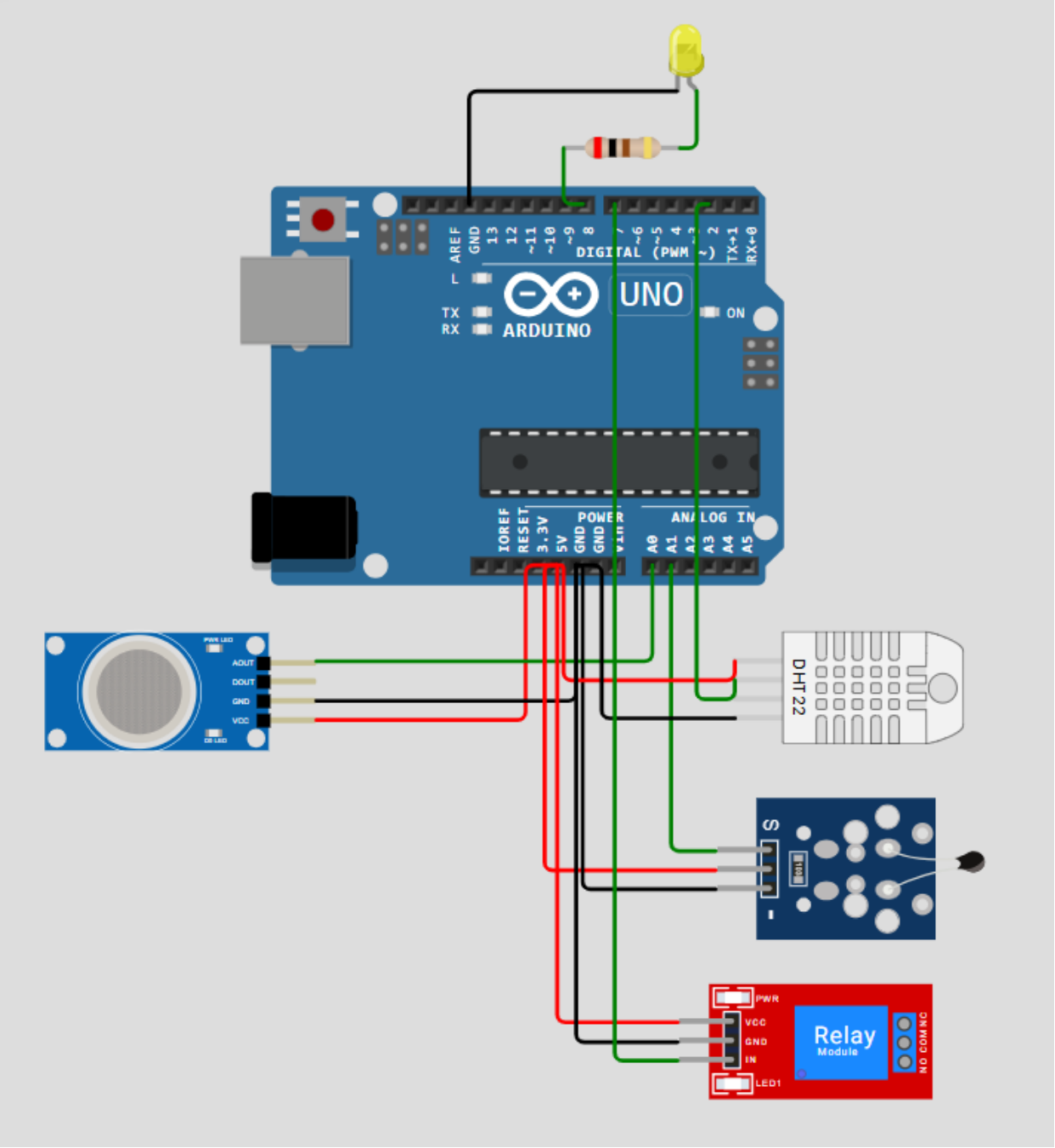


Рисунок Г.1 — Предоставленный скриншот модели Wokwi

Проект: https://wokwi.com/projects/479217730910433281.

Приложение Д. Листинг программы Arduino

```
// Вариант 17. Кухня ресторана. Контроль загазованности
// Плата: Arduino Uno (ATmega328P), симулятор Wokwi
// Подключение:
//   Датчик 1 (MQ2, газ 0-100 %)        -> AOUT на A0
//   Датчик 2 (NTC, температура, °C)    -> OUT на A1
//   Датчик 3 (DHT22, влажность, %)     -> SDA на D2
//   Реле K1 (вытяжка)                  -> IN на D7
//   Светодиод HL1 (жёлтый) через R1 = 200 Ом -> D8
#include <DHT.h>
const int PIN_S1    = A0;   // датчик 1: MQ2 (газ)
const int PIN_S2    = A1;   // датчик 2: NTC (температура)
const int PIN_S3    = 2;    // датчик 3: DHT22 (влажность)
const int PIN_RELAY = 7;    // вытяжка
const int PIN_LED   = 8;    // светодиод-индикатор
const unsigned long T_MS = 1000;  // период опроса T = 1 с
const float S1_MIN = 0.0;         // диапазон датчика 1: 0...100 %
const float S1_MAX = 100.0;
const float THRESHOLD = 40.0;     // порог: газ > 40 %
const float BETA = 3950.0;        // коэффициент NTC в Wokwi
DHT dht(PIN_S3, DHT22);
unsigned long n = 0;   // номер измерения
bool alarm = false;    // порог сейчас превышен?
// Датчик 1: MQ2 (газ, %)
float readS1() {
  return S1_MIN + (S1_MAX - S1_MIN) * analogRead(PIN_S1) / 1023.0;
}
// Датчик 2: NTC (температура, °C)
float readS2() {
  int raw = analogRead(PIN_S2);
  if (raw < 1) raw = 1;  // защита от деления на ноль
  return 1.0 / (log(1.0 / (1023.0 / raw - 1.0)) / BETA + 1.0 / 298.15) - 273.15;
}
// Датчик 3: DHT22 (влажность, %)
float readS3() {
  return dht.readHumidity();
}
void actuatorOn()  { digitalWrite(PIN_RELAY, HIGH); }
void actuatorOff() { digitalWrite(PIN_RELAY, LOW); }

bool conditionMet(float s1, float s2, float s3) {
  return s1 > THRESHOLD;
}
void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  actuatorOff();
  digitalWrite(PIN_LED, LOW);
}
void loop() {
  n++;
  float s1 = readS1();
  float s2 = readS2();
  float s3 = readS3();
Serial.print("DATA n="); Serial.print(n);
  Serial.print(" s1="); Serial.print(s1, 2);
  Serial.print(" s2="); Serial.print(s2, 1);
  Serial.print(" s3=");
  if (isnan(s3)) Serial.println("nan"); else Serial.println(s3, 1);
  bool cond = conditionMet(s1, s2, s3);
  if (cond && !alarm) {
    alarm = true;
    actuatorOn();
    digitalWrite(PIN_LED, HIGH);
    Serial.print("EVENT n="); Serial.print(n);
    Serial.println(" type=THRESHOLD");
  } else if (!cond && alarm) {
    alarm = false;
    actuatorOff();
    digitalWrite(PIN_LED, LOW);
    Serial.print("EVENT n="); Serial.print(n);
    Serial.println(" type=NORMAL");
  }
  delay(T_MS);
}
```

Приложение Е. Листинг функции Node-RED

function из nodered/flow.json; начало

```
const T = 1;

const RANGES = {
    s1: [0, 100],
    s2: [-20, 80],
    s3: [0, 100]
};

const text = String(msg.payload).replace(/^\uFEFF/, "");
const sql = [];

try {
    for (const raw of text.split(/\r?\n/)) {
        const line = raw.trim();
        if (!line) continue;

        const parts = line.split(/\s+/);
        const kind = parts.shift();

        // Пропускаем служебные сообщения симулятора.
        if (kind !== "DATA" && kind !== "EVENT") continue;

        const f = {};
        for (const part of parts) {
            const i = part.indexOf("=");
            if (i > 0) {
                f[part.slice(0, i)] = part.slice(i + 1);
            }
        }

        const n = Number(f.n);
        if (!Number.isSafeInteger(n) || n < 1) {
            throw new Error("Некорректный n: " + line);
        }

        const t = n * T;

        if (kind === "DATA") {
            for (const sensor of ["s1", "s2", "s3"]) {
                if (f[sensor] === undefined || f[sensor] === "") {
                    throw new Error("Нет " + sensor + ": " + line);
                }

                const value = Number(f[sensor]);
                if (!Number.isFinite(value)) {
                    throw new Error("Не число " + sensor + ": " + line);
                }

                sql.push(
                    `INSERT INTO readings(n,t_s,sensor,value) ` +
                    `VALUES(${n},${t},'${sensor}',${value});`
                );
```

Приложение Е (продолжение)

```
                const [min, max] = RANGES[sensor];

                if (value < min || value > max) {
                    sql.push(
                        `INSERT INTO events(n,t_s,type,message) ` +
                        `VALUES(${n},${t},'FAULT',` +
                        `'датчик ${sensor} вне диапазона');`
                    );
                }
            }
        } else {
            if (!["THRESHOLD", "NORMAL"].includes(f.type)) {
                throw new Error("Неизвестный тип события: " + line);
            }

            sql.push(
                `INSERT INTO events(n,t_s,type,message) ` +
                `VALUES(${n},${t},'${f.type}','');`
            );
        }
    }

    if (sql.length === 0) {
        throw new Error("В файле нет строк DATA или EVENT");
    }
} catch (err) {
    node.error(err.message, msg);
    return null;
}

const head =
    "BEGIN;\n" +
    "DELETE FROM readings;\n" +
    "DELETE FROM events;\n" +
    "DELETE FROM sqlite_sequence " +
    "WHERE name IN ('readings','events');\n";

return [
    null,
    {
        payload: head + sql.join("\n") + "\nCOMMIT;\n"
    }
];
```

Приложение Ж. Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings (
  id     INTEGER PRIMARY KEY AUTOINCREMENT,
  n      INTEGER NOT NULL,
  t_s    INTEGER NOT NULL,
  sensor TEXT    NOT NULL,
  value  REAL    NOT NULL
);

CREATE TABLE IF NOT EXISTS events (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  n       INTEGER NOT NULL,
  t_s     INTEGER NOT NULL,
  type    TEXT    NOT NULL,
  message TEXT
);
```

db/queries.sql

```
-- 1. 10 последних показаний
SELECT * FROM readings ORDER BY id DESC LIMIT 10;

-- 2. минимум, среднее и максимум по каждому датчику
SELECT sensor, MIN(value) AS min_v, ROUND(AVG(value), 2) AS avg_v, MAX(value) AS max_v
FROM readings GROUP BY sensor;

-- 3. все события по порядку
SELECT * FROM events ORDER BY id;
```

Приложение И. Журнал Serial и загрузочный скрипт

data/log.txt — предоставленный фрагмент штатного сеанса

```
DATA n=9 s1=47.90 s2=24.0 s3=50.0
DATA n=10 s1=27.08 s2=24.0 s3=50.0
EVENT n=10 type=NORMAL
DATA n=11 s1=27.08 s2=24.0 s3=50.0
DATA n=12 s1=66.57 s2=24.0 s3=50.0
EVENT n=12 type=THRESHOLD
DATA n=13 s1=62.66 s2=24.0 s3=50.0
DATA n=14 s1=50.73 s2=24.0 s3=50.0
DATA n=15 s1=72.92 s2=24.0 s3=50.0
DATA n=16 s1=30.50 s2=24.0 s3=50.0
EVENT n=16 type=NORMAL
DATA n=17 s1=83.97 s2=24.0 s3=50.0
EVENT n=17 type=THRESHOLD
DATA n=18 s1=83.97 s2=24.0 s3=50.0
```

data/log_fault.txt — тест выхода за диапазон

```
DATA n=1 s1=999 s2=25.0 s3=50.0
```

db/load.sql — скрипт для аварийного теста

```
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN ('readings','events');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,1,'s1',999);
INSERT INTO events(n,t_s,type,message) VALUES(1,1,'FAULT','датчик s1 вне диапазона');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,1,'s2',25);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,1,'s3',50);
COMMIT;
```

Результат программного воспроизведения: 3 показания и 1 событие FAULT. Для возврата к штатной выборке нужно снова загрузить data/log.txt.

Приложение К. История Git

Фактический вывод git log --oneline из предоставленного архива

```
d953205 Правки описания проекта
a12eb6d Заполнено описание проекта и инструкция запуска
dd5beeb Проверено обнаружение неисправности датчика s1
7162fa9 Настроена загрузка данных из лога через Node-RED в SQLite
7105b6a Схема EasyEDA, BOM и структурная схема
bb295b3 ER-диаграмма и контрольные запросы
dc0a261 Wokwi: ссылка в README, скриншот схемы
3eff7a8 Структура проекта, README, .gitignore
ed510a1 Схема БД schema.sql и контрольные запросы
8635051 Тестовый файл неисправности log_fault.txt
bc1a5ff Добавлена прошивка variant_17.ino
27b9574 Добавлена схема в Wokwi (вариант 17)
```

Дата всех перечисленных коммитов — 29.10.2026. Порядок вывода: от последнего изменения к первому. В отчёте не используются условные хеши и даты из PDF-образца.

Командный файл start-nodered.bat

```
@echo off
rem Starts Node-RED from the project folder.
set TOOLS=%USERPROFILE%\tools
set PATH=%TOOLS%\node;%TOOLS%\sqlite;%PATH%
cd /d %~dp0
node %TOOLS%\nodered\node_modules\node-red\red.js --userDir %TOOLS%\nodered-data
pause
```